

Arrays and Array-Based Techniques (Two-Pointer, Sliding Window, Prefix Sums)

LAB WORKBOOK-24.07.2026

NAME : FEBINE M E

REG NO : 25BAD024

1. Squaring a Sorted Array (Two-Pointer)

AIM :

To calculate the kinetic energy of these waves, she needs to square each value and return a new array containing the squares, sorted in non-decreasing order.

PSEUDO CODE

BEGIN

Read N

Read array arr

Create result array

left $\leftarrow$ 0

right $\leftarrow$ N - 1

index $\leftarrow$ N - 1

WHILE left <= right **DO**

IF |arr[left]| > |arr[right]| **THEN**

$\text{result}[\text{index}] \leftarrow \text{arr}[\text{left}] \times \text{arr}[\text{left}]$

$\text{left} \leftarrow \text{left} + 1$

ELSE

$\text{result}[\text{index}] \leftarrow \text{arr}[\text{right}] \times \text{arr}[\text{right}]$

$\text{right} \leftarrow \text{right} - 1$

END IF

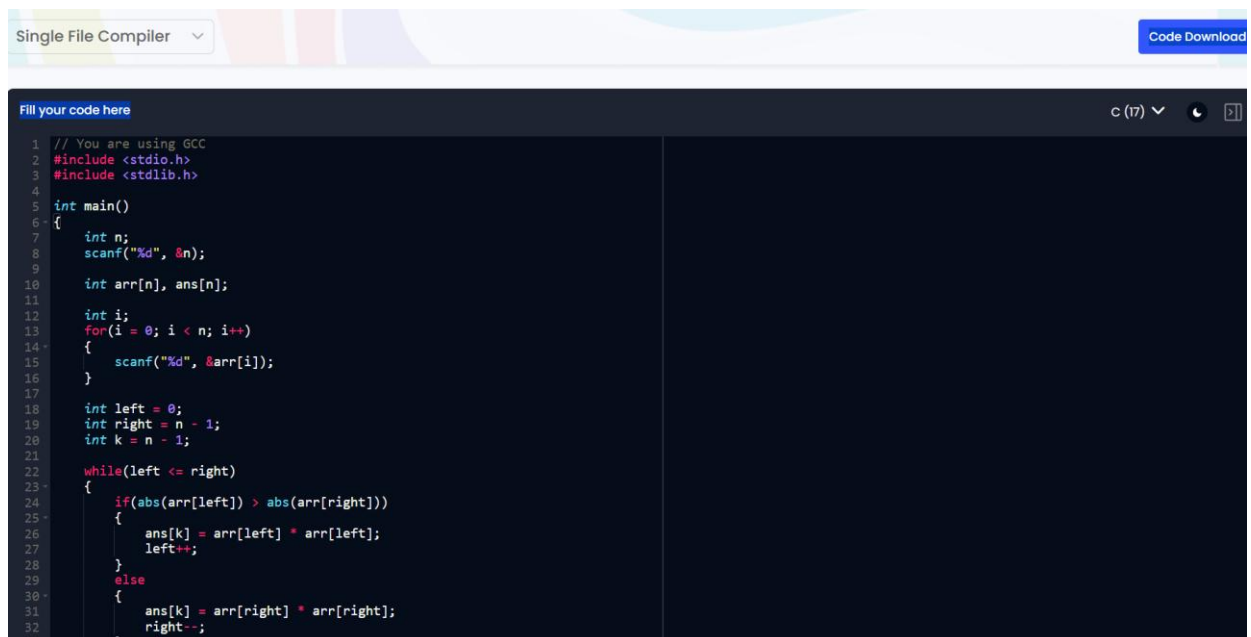
$\text{index} \leftarrow \text{index} - 1$

END WHILE

Print result array

END

CODE :



```
1 // You are using GCC
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 int main()
6 {
7     int n;
8     scanf("%d", &n);
9
10    int arr[n], ans[n];
11
12    int i;
13    for(i = 0; i < n; i++)
14    {
15        scanf("%d", &arr[i]);
16    }
17
18    int left = 0;
19    int right = n - 1;
20    int k = n - 1;
21
22    while(left <= right)
23    {
24        if(abs(arr[left]) > abs(arr[right]))
25        {
26            ans[k] = arr[left] * arr[left];
27            left++;
28        }
29        else
30        {
31            ans[k] = arr[right] * arr[right];
32            right--;
33        }
34    }
```

```

18 int left = 0;
19 int right = n - 1;
20 int k = n - 1;
21
22 while(left <= right)
23 {
24     if(abs(arr[left]) > abs(arr[right]))
25     {
26         ans[k] = arr[left] * arr[left];
27         left++;
28     }
29     else
30     {
31         ans[k] = arr[right] * arr[right];
32         right--;
33     }
34     k--;
35 }
36
37 for(i = 0; i < n; i++)
38 {
39     printf("%d ", ans[i]);
40 }
41
42 return 0;
43 }

```

OUTPUT :

```

5
-4 -10 3 10

```

Compiler Message

Compilation successful

Custom Testcase

Output

```
0 1 9 16 100
```

2. Cumulative Resource Consumption (Prefix Sum)

AIM :

To implement the Prefix Sum algorithm in C to calculate the cumulative sum of the elements in an array.

PSEUDO CODE :

BEGIN

Read N

Read array arr

Create array prefix

$\text{prefix}[0] \leftarrow \text{arr}[0]$

FOR i $\leftarrow$ 1 **TO** N - 1 **DO**

$\text{prefix}[i] \leftarrow \text{prefix}[i - 1] + \text{arr}[i]$

END FOR

Print prefix array

END

CODE :

Fill your code here

```
1  #include <stdio.h>
2
3  int main()
4  {
5      printf("FEBINE M E\n");
6      printf("25BAD024\n\n");
7      int n;
8      scanf("%d", &n);
9
10     int arr[n], prefix[n];
11
12
13     for(int i = 0; i < n; i++)
14     {
15         scanf("%d", &arr[i]);
16     }
17
18     prefix[0] = arr[0];
19
20
21     for(int i = 1; i < n; i++)
22     {
23         prefix[i] = prefix[i - 1] + arr[i];
24     }
25
26
27     for(int i = 0; i < n; i++)
28     {
29         printf("%d ", prefix[i]);
30     }
31
32     return 0;
33 }
34
```

OUTPUT :

5
10 20 5 15 30

Compiler Message

Compilation successful

Custom Testcase

Output

FEBINE M E
25BAD024
10 30 35 50 80

3. Grouping Active Sectors (Fixed-Size Sliding Window)

AIM :

To implement the Fixed-Size Sliding Window algorithm in C to determine the minimum number of swaps required to bring all 1s together in a binary array.

PSEUDO CODE :

BEGIN

Read N

Read array arr

count $\leftarrow$ Number of 1's in arr

IF count $\leq$ 1 THEN

Print "Min Swaps: 0"

STOP

END IF

ones $\leftarrow$ Number of 1's in the first window of size count

maxOnes $\leftarrow$ ones

FOR each remaining window DO

IF left element is 1 THEN

ones $\leftarrow$ ones - 1

END IF

IF new right element is 1 THEN

ones $\leftarrow$ ones + 1

END IF

IF ones > maxOnes THEN

maxOnes $\leftarrow$ ones

END IF

END FOR

swaps $\leftarrow$ count - maxOnes

Print "Min Swaps:", swaps

END

CODE:

Fill your code here

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int n;
6      scanf("%d", &n);
7
8      int arr[n];
9
10
11     for(int i = 0; i < n; i++)
12     {
13         scanf("%d", &arr[i]);
14     }
15
16
17     int count = 0;
18     for(int i = 0; i < n; i++)
19     {
20         if(arr[i] == 1)
21         {
22             count++;
23         }
24     }
25
26
27     if(count <= 1)
28     {
29         printf("Min Swaps: 0");
30         return 0;
31     }
32
33
```

Fill your code here

```
34
35     int ones = 0;
36     for(int i = 0; i < count; i++)
37     {
38         if(arr[i] == 1)
39         {
40             ones++;
41         }
42     }
43
44     int max = ones;
45
46     // Move the window
47     for(int i = count; i < n; i++)
48     {
49         if(arr[i - count] == 1)
50         {
51             ones--;
52         }
53
54         if(arr[i] == 1)
55         {
56             ones++;
57         }
58
59         if(ones > max)
60         {
61             max = ones;
62         }
63     }
64
65     printf("Min Swaps: %d", count - max);
```

OUTPUT :

6
101010

Compiler Message

Compilation successful

Custom Testcase

Output

FEBINE M E
25BAD024

Min Swaps: 1

4. Maximum Task Window Within Budget (Variable-Size Sliding Window)

PSEUDO CODE:

BEGIN

Read N, P

Read array arr

start $\leftarrow$ 0

sum $\leftarrow$ 0

maxLength $\leftarrow$ 0

FOR end $\leftarrow$ 0 TO N - 1 DO

sum $\leftarrow$ sum + arr[end]

WHILE sum > P DO

sum $\leftarrow$ sum - arr[start]

start $\leftarrow$ start + 1

END WHILE

IF (end - start + 1) > maxLength THEN

maxLength $\leftarrow$ end - start + 1

END IF

END FOR

Print "Max Length:", maxLength

END

CODE:

Fill your code here

```
1  #include <stdio.h>
2
3  int main()
4  {
5      printf("FEBINE M E\n");
6      printf("25BAD024\n\n");
7
8
9      int n, p;
10     scanf("%d %d", &n, &p);
11
12     int arr[n];
13
14     for(int i = 0; i < n; i++)
15     {
16         scanf("%d", &arr[i]);
17     }
18
19     int start = 0;
20     int sum = 0;
21     int max = 0;
22
23     for(int end = 0; end < n; end++)
24     {
25         sum = sum + arr[end];
26
27         while(sum > p)
28         {
29             sum = sum - arr[start];
30             start++;
31         }
32
33         if(end - start + 1 > max)
```

```
31     }
32
33     if(end - start + 1 > max)
34     {
35         max = end - start + 1;
36     }
37 }
38
39 printf("Max Length: %d", max);
40
41 return 0;
42 }
43
44 return 0;
45 }
```

OUTPUT :

```
5 8
3 1 2 4 5
```

Compiler Message

Compilation successful

Custom Testcase

Output

```
FEBINE M E
25BAD024
```

Max Length: 3

5. Sliding Window Maximum Peak (Monotonic Window Queue)

PSEUDO CODE:

BEGIN

Read N, K

Read array arr

Create an empty deque

FOR $i \leftarrow 0$ TO $N - 1$ DO

WHILE deque is not empty AND

arr[back of deque] $\leq$ arr[i] DO

Remove the back element from deque

END WHILE

Insert i at the back of deque

IF front of deque $< i - K + 1$ THEN

Remove the front element from deque

END IF

IF $i \geq K - 1$ THEN

Print arr[front of deque]

END IF

END FOR

END

CODE:

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("FEBINE M E\n");
6     printf("25BAD024\n\n");
7     int n, k;
8     scanf("%d %d", &n, &k);
9
10    int arr[n];
11    int dq[n];
12    int front = 0;
13    int rear = -1;
14
15    for(int i = 0; i < n; i++)
16        scanf("%d", &arr[i]);
17
18    for(int i = 0; i < n; i++)
19    {
20        while(front <= rear && arr[dq[rear]] <= arr[i])
21            rear--;
22
23        dq[++rear] = i;
24
25        if(dq[front] <= i - k)
26            front++;
27
28        if(i >= k - 1)
29            printf("%d ", arr[dq[front]]);
30    }
31
32    return 0;
33 }
34
```

OUTPUT:

```
83
13-1-35367
```

Compiler Message

Compilation successful

Custom Testcase

Output

```
FEBINE M E
25BAD024
```

```
3 3 5 5 6 7
```